Who Uses Developer Communities? Collaboration Trajectories between Developers and Agents in Agentic Coding

ANONYMOUS AUTHOR(S)

AI coding changes how developers use technical communities. Rather than independently searching documentation, examples, and community discussions, developers increasingly delegate retrieval, code drafting, tool use, and iteration to agents. Yet an agent's apparent autonomy does not remove the developer's responsibility to state goals, supply local facts, assess evidence, and validate outcomes. We introduce the Developer–Agent Collaboration Mechanism (DACM), a framework that treats agentic coding as a feedback loop among developers, coding agents, technical knowledge ecosystems, and local engineering environments. DACM is grounded in observable agent work: context construction, retrieval, candidate action, tool feedback, and iteration. We present a sequential study design that uses a formative audit of 26 matched CANN/CUDA development tasks to construct knowledge conditions, followed by a planned qualitative study with approximately 20 developers. The study combines retrospective replay of recent AI-assisted work with a common operator-development scenario completed in participants' own IDEs, CLIs, and agents. We propose multi-actor collaboration trajectory analysis to trace evidence acquisition, fact requests, human takeover, verification checkpoints, and stopping points. The contribution is not a model benchmark or a hardware comparison; it is an account of how technical communities become infrastructure for a developer–agent collective, and how their design shapes whether collaborative work can advance with accountable evidence.

CCS Concepts: • **Human-centered computing** → **User studies**; • **Software and its engineering** → *Software development process management*.

Additional Key Words and Phrases: human–AI collaboration; AI coding; coding agents; developer experience; technical documentation; developer communities

1 INTRODUCTION

When developers encounter an unfamiliar API, compiler error, version conflict, or performance problem, they have traditionally searched, read documentation, compared community answers, executed code locally, and adjusted their next step. AI coding tools reorganize this path. Developers can now describe a goal or paste an error, while an agent retrieves documentation, reads project files, proposes patches, invokes tools, observes feedback, and iterates. This does not make developers passive approvers. They still determine goals and boundaries, decide what context and permissions to provide, judge risk, and remain responsible for the delivered result.

The change is therefore not simply that "AI does more." Coding agents operate through a bounded loop: they construct context from a task and available artifacts, retrieve or read evidence, propose an action, call tools when permitted, and revise in response to feedback. They cannot reliably infer facts that are unavailable to them: an actual version combination, a hardware topology, private code semantics, a business constraint, or an acceptance threshold. Moreover, readable prose or fluent generated code does not ensure that an instruction is applicable to the local environment. A phase in which an agent generates much of the visible output may still impose substantial verification work on a developer.

This shift also changes the role of a technical community. Documentation, examples, compatibility matrices, repositories, issue trackers, and forums are no longer only materials that a human reads directly. They are evidence infrastructure that an agent must discover, access, interpret, cite, and turn into a bounded action. A page being visible in a

2026. Manuscript submitted to ACM

browser does not mean that an agent can obtain its relevant body text; an answer containing plausible steps does not mean that its prerequisites and versions are established. Assessing only answer quality obscures whether a failure arose from a model, a retrieval path, documentation engineering, version governance, a thin community, or missing local facts. Conversely, assessing only conventional developer-portal usability cannot explain why developers repeatedly search, correct, and verify after an agent has suggested a solution.

We introduce the *Developer–Agent Collaboration Mechanism* (DACM) to study this situation. DACM models development as a loop among four actors: a developer, an agent, a technical knowledge ecosystem, and a local engineering environment. It uses an agent's observable workflow as an analytic foundation, without making agent internals or a new model architecture the paper's main claim. Our research question is:

> *How do developers, agents, technical knowledge ecosystems, and local environments jointly advance a development task in agentic coding?*

We use a sequential design. Study 0 is a formative audit of 26 result-matched CANN/CUDA development tasks. It observes a web-enabled agent's access to official and third-party knowledge, version evidence, retrieval cost, and actionability. The audit is not used to claim which ecosystem is globally superior; it identifies task settings with meaningfully different knowledge conditions. Study 1 then replays participants' recent AI-assisted development work. Study 2 asks the same participants to work through a common AddCustom-to-PyTorch scenario in their own familiar IDE, CLI, browser, and agent. The scenario does not require an accelerator card or a CANN installation; it offers starter code, environment manifests, evidence excerpts, and staged logs so that the study can focus on collaboration with knowledge and local evidence rather than hardware performance.

This paper contributes:

(1) DACM, a framework linking an agent's observable context–retrieval–action–feedback loop to developers' goal setting, fact provision, judgement, and validation;
(2) multi-actor collaboration trajectory analysis, which represents development as ordered events among people, agents, knowledge sources, and local environments rather than as a final answer or a satisfaction score;
(3) a research design that uses a formative knowledge-ecosystem audit to construct, but not predetermine, qualitative collaboration studies; and
(4) design implications for developer communities as evidence infrastructure for a developer–agent collective.

2 RELATED WORK

2.1 Developers Working with Generative AI

Research on generative programming tools shows that developers use generated code as a draft, a conversational partner, a search shortcut, and a vehicle for rapid experimentation. They also inspect diffs, run tests, seek external evidence, and constrain permissions to manage risk [1, 2]. These studies make clear that adoption is not a binary decision. DACM extends this work by considering the external technical knowledge that an agent needs in order to act, and the local facts that only a developer or environment can provide.

2.2 Agentic Workflows and Their Boundaries

Agents differ from one-shot code completion because they may interleave reasoning, retrieval, file inspection, editing, and tool use [3, 4]. This loop can transform static knowledge into candidate actions, but it is bounded by available context, accessible evidence, permissions, and feedback. We therefore treat the loop as an observable interactional

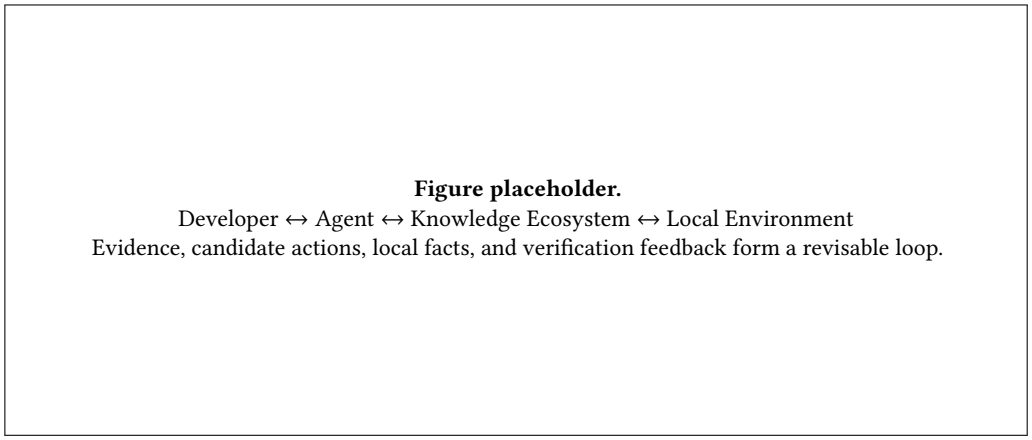

Fig. 1. The DACM loop. The final version will replace this accessible placeholder with a vector diagram and descriptive alt text.

structure: not a claim about how every model internally reasons, but a basis for observing when a developer must supply facts, choose a source, accept a proposal, or take over.

2.3 Developer Knowledge Practices and Communities

Documentation, examples, search, Q&A sites, and peer networks shape how developers learn and troubleshoot. In agentic coding, these resources also function as sources and validators for an agent. We call the combined supply of official documentation, examples, version information, repositories, and third-party discussion a *technical knowledge ecosystem*. This term foregrounds a design question that conventional portal evaluation leaves open: can knowledge be found, accessed, attributed, and acted upon by a developer and an agent together?

3 DACM: DEVELOPER–AGENT COLLABORATION MECHANISM

DACM contains four actors and a feedback loop. Developers define objectives and constraints, decide what may be shared or executed, evaluate risk, and remain accountable. Agents construct context, retrieve and compare sources, propose plans or code, call permitted tools, and interpret feedback. Technical knowledge ecosystems provide official documentation, examples, version information, repositories, and community discussions. Local environments contain code, dependencies, hardware, configuration, data, terminal output, and tests; they are where constraints and final validation occur.

The framework is operationalized through five dimensions (Table 1). These dimensions are derived from the observable work of coding agents, but each retains a developer role. For example, an agent may read a compatibility page, but a developer must determine whether its version range applies to the current project; an agent may execute a test, but a developer may need to decide whether the observed result is acceptable.

The minimal analytic unit is a *collaboration event*: time, initiator, action, input evidence, output state, transfer of control, and next destination. A statement such as "the agent suggested an upgrade" is not sufficient evidence on its own. Analysis must also ask what source supported the suggestion, whether the developer supplied a version matrix, whether an action was executed, how feedback was returned, and who chose to stop.

Table 1. DACM dimensions derived from observable agent workflows.

Dimension	Typical agent work	Developer work that remains consequential
Goal and context	Read task, files, and prior dialogue; ask for clarification	Set priorities; disclose constraints; decide what can be shared
Knowledge acquisition	Search, read documentation, inspect repositories, compare sources	Judge relevance and applicability to the project
Candidate action	Draft code, commands, plans, or diagnostic hypotheses	Accept, reject, constrain scope, or revise a proposal
Local execution	Invoke permitted tools and interpret their outputs	Supply environment state and manage high-risk or unavailable actions
Verification and feedback	Revise after logs, tests, or new evidence	Determine adequacy against quality and business criteria

We define collaboration effectiveness as a process property rather than an automation ratio. A segment is more effective when it advances the task with traceable evidence, makes missing local facts explicit, avoids unproductive repetition, and reaches a verification checkpoint proportionate to risk. We will examine four outcomes: progress to a next actionable state, evidence sufficiency, clarity of handoff, and reachability of verification.

4 FORMATIVE KNOWLEDGE-ECOSYSTEM AUDIT (STUDY 0)

Study 0 constructs the empirical conditions for the participant studies. It covers 26 development goals across installation, operator development, training, inference and deployment, performance optimization, debugging, and migration. For each goal, we formulate a question in CANN and CUDA using each ecosystem's actual terminology while keeping the desired user outcome comparable. A single web-enabled agent follows a common protocol: search, record source changes, read core official material, inspect version evidence, seek third-party support, and assess whether the eventual answer can be bounded and acted on.

The audit uses 11 indicators covering official discoverability and accessibility, content detail, version clarity, third-party support, model prior knowledge, retrieval cost, and actionability. It is deliberately kept separate from the developer-side journey audit. The former describes conditions under which an agent can obtain task knowledge; the latter describes a human developer's direct portal experience. Neither becomes a universal score for an ecosystem.

Study 0 produces three formative assets: task-condition cards, a continuous AddCustom-to-PyTorch scenario, and paired knowledge conditions. One condition offers a relatively continuous, readable, and version-bounded evidence chain. The other requires source stitching or contains gaps in version information, body accessibility, or local facts. These conditions are not designed to make participants fail. They let us observe how developers and agents identify unknowns and formulate verification when direct accelerator execution is unavailable.

5 METHOD: STUDYING COLLABORATION TRAJECTORIES

5.1 Participants and Procedure

We plan to recruit approximately 20 developers who have used an AI coding tool in real work within the prior three months. Sampling will seek variation in role, experience, IDE/CLI practices, agents, and task domains rather than

statistical representativeness. Participants need not own an Ascend device, maintain a CANN environment, or disclose an entire private repository. They may redact, skip, or remove any material.

Each session has five parts: consent and background (10 minutes); retrospective replay of a recent AI-assisted task (35–45 minutes); a common scenario task (50–60 minutes); replay interview and short reflection (15–20 minutes); and consent confirmation (5 minutes). Study 1 uses available anchors such as an agent conversation, terminal excerpt, diff, issue, pull request, document visit, screenshot, or participant narration. The interviewer follows temporal events instead of asking participants to produce an artificial plan.

In Study 2, participants work in their own tools with a lightweight scenario package. The five connected stages are: (D) writing a kernel, (M) tiling, (X) compile/memory evidence, (O) registration and framework integration, and (L) accuracy acceptance. Starter code, manifests, document links, excerpts, and staged logs provide a credible engineering context without falsely simulating accelerator execution. Participants may search, ask their usual agent, edit code or notes, run safe local scripts, articulate uncertainty, and state what they would verify in a real environment. The researcher only manages time and recording boundaries.

5.2 Data and Analysis

With consent, we will collect interview audio, necessary screen capture, agent dialogue, terminal commands and output, source URLs, code diffs, researcher field notes, and a post-task replay. Data will be de-identified; raw recordings, transcripts, event tables, and quotable excerpts will be access controlled separately.

We will combine multi-actor collaboration trajectory analysis with reflexive thematic analysis. A trajectory event records its initiator, action, evidence, knowledge state, lead actor, handoff target, and result. Initial codes include goal setting, context provision, official/third-party/project knowledge, generation, retrieval, editing, execution, fact request, source switching, human takeover, verification, rollback, and stopping. Two researchers will first jointly code a subset, revise definitions through discussion, then compare cases iteratively. Themes will be supported by participant accounts, trajectory events, and available artifacts; Study 0 scores will not be used to infer developer behavior.

6 PLANNED FINDINGS AND EVIDENCE STANDARD

[**TODO:** This section must be replaced with evidence after Study 1 and Study 2. It is deliberately not written as completed findings.]

We expect the analysis to examine four questions rather than to confirm predetermined claims: (1) when agents compress information seeking into candidate action, and when they add verification overhead; (2) how developers recognize and supply local facts; (3) how knowledge conditions alter the shape of trajectories rather than merely a single answer; and (4) how developers describe their continuing roles in boundary setting, evidence judgement, and responsibility. Each final finding will include cross-case patterning, anonymized quotations, a trajectory fragment, and a counterexample or boundary condition.

7 DESIGN IMPLICATIONS

7.1 Publish Actionable Evidence Units

Documentation should support a task with a goal, applicable versions, prerequisites, minimal commands or code, expected output, common failures, recovery paths, and a stable source. Such units let agents cite rather than stitch unsupported instructions, and let developers assess applicability quickly.

7.2 Make Version Relations Queryable Constraints

Compatibility among drivers, firmware, toolkits, frameworks, plugins, and hardware is often the earliest local fact required in collaboration. Communities should provide deep-linkable, machine-readable compatibility matrices and minimal environment manifests. Agents should request missing facts explicitly instead of silently guessing.

7.3 Design for Safe Fact Return and Verification

Developers cannot always expose an entire environment, but they can provide redacted version summaries, log excerpts, dependency trees, test results, and error codes. Tools should make it clear why an agent needs a fact and what decision it changes. For risky or version-sensitive changes, systems should expose an evidence and verification checkpoint before automatic execution.

8 DISCUSSION AND LIMITATIONS

Technical communities now face a composite user: a developer and an agent jointly enter, read, organize, and verify technical knowledge. The developer remains the accountable actor, but their work may concentrate in goal setting, fact provision, risk judgement, validation, and closure. The agent may carry much of the retrieval, compression, drafting, and local action. Whether this division is useful depends on whether knowledge can flow as reliable evidence.

This study has clear limits. Study 0 fixes one agent and tool configuration and cannot represent all models or enterprise search systems. The CANN/CUDA audit is formative; it is not a permanent ranking of ecosystems or hardware. A qualitative study of approximately 20 participants is suited to mechanism discovery rather than population estimates or causal effects. Finally, the common scenario does not substitute for longitudinal, card-enabled, team-based development. These limitations motivate future field studies, comparative agent studies, and interventions that test documentation and tool designs against recurrent collaboration breakdowns.

9 CONCLUSION

DACM frames agentic coding as a collaboration among developers, agents, technical knowledge ecosystems, and local environments. By tracing goals, evidence, action, local facts, handoffs, and verification, it shifts the question from whether AI is generically useful to how collaboration can advance with accountable evidence. The proposed sequential study will establish how developer communities can serve not only human readers, but developer–agent collectives.

REFERENCES

[1] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. *Proceedings of the ACM on Human-Computer Interaction* 7, CSCW1 (2023).

[2] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*. ACM.

[3] John Yang, Carlos E. Jimenez, Alexander Wettig, et al. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*.

[4] Shunyu Yao, Jeffrey Zhao, Dian Yu, et al. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.